Proximal Policy Optimization Algorithms

John Schulman Filip Wolski Prafulla Dhariwal Alec Radford
Oleg Klimov

OpenAI

Not published in any peer reviewed venue! Just ArXiv submission!

Western
UNIVERSITY · CANADA

Table of Contents

Policy Gradient Estimator

Policy gradient methods compute an estimator of the policy gradient and plug it into a stochastic gradient ascent algorithm. The fundamental gradient estimator is:

Gradient Estimator

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \hat{A}_t \right]$$

Key Components:

- $\hat{\mathbb{E}}_t$: Empirical expectation over a finite batch of samples.
- π_θ: The stochastic policy parameterized by θ.
- $\nabla_\theta \log \pi_\theta(a_t|s_t)$: The direction to shift the parameters to increase the probability of taking action a_t in state s_t.
- $\hat{A}_t$: Estimator of the advantage function (measures how much better action a_t is compared to the average policy behavior at s_t).

Western UNIVERSITY · CANADA

Policy Gradient Estimator - Deep Dive

- Why do we take gradient of log policy? Why not gradient of policy?

A Quick Refresher: What is an Expectation?

An **expectation** (or expected value) is simply a weighted average of all possible outcomes, where the weights are the probabilities of those outcomes occurring.

For a discrete variable x drawn from a probability distribution P, the expected value of a function $f(x)$ is defined as:

Definition of Expectation

$$\mathbb{E}_{x\sim P}[f(x)] = \sum_x P(x)f(x)$$

Breaking it down:

- $P(x)$: The *probability weight* (how likely the outcome is).
- $f(x)$: The *value* of that outcome.
- The expectation operator $\mathbb{E}$ is literally just a summation!

Supervised Learning - Fixed Data Distribution

1. The Objective as a Sum

Let $P(x, y)$ be the true, static probability of drawing that specific image-label pair from the dataset (which is usually just $\frac{1}{N}$ for a dataset of size N).

$$J(\theta) = \sum_{x,y} P(x, y)\text{Loss}(f_\theta(x), y)$$

2. Taking the Gradient

Now, we want to update our network parameters θ, so we take the gradient of this objective:

$$\nabla_\theta J(\theta) = \nabla_\theta \left[\sum_{x,y} P(x, y)\text{Loss}(f_\theta(x), y) \right]$$

Supervised Learning - Fixed Data Distribution

Because the summation is linear, the gradient can move inside.

Here is the crucial difference from RL: **The data distribution $P(x, y)$ has absolutely no dependency on your neural network's weights θ.** Your network's parameters do not change what images are in the dataset. Therefore, with respect to θ, $P(x, y)$ is just a constant.

The gradient simply passes right over it and only hits the loss function:

$$\nabla_\theta J(\theta) = \sum_{x,y} P(x, y) \nabla_\theta \text{Loss}(f_\theta(x), y)$$

3. Expectation

We still have the exact mathematical structure of an expectation: $\sum P(x, y) \times$ [something]. Because the original probability weight $P(x, y)$ survived the derivative completely untouched, we can effortlessly rewrite this sum into an expectation format:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,y \sim \text{Dataset}}[\nabla_\theta \text{Loss}(f_\theta(x), y)]$$

So we can do stochastic optimization without calculating gradient over the entire dataset!

Reinforcement Learning Objective - Moving Data

The Objective as a Sum

Unrolling the expected reward objective into a sum over all possible trajectories (τ):

$$J(\theta) = \sum_{\tau} P(\tau|\theta)R(\tau)$$

The Critical Difference:

- $P(\tau|\theta)$ is the probability of experiencing trajectory τ. Unlike the fixed dataset, this probability is **strictly dependent on your neural network parameters** θ.
- $R(\tau)$ is the total reward for that trajectory, which is determined entirely by the rules of the environment.

Reinforcement Learning Objective - Moving Data

When we take the gradient, it directly hits the probability distribution because it depends on θ:

$$\nabla_\theta J(\theta) = \sum_\tau \nabla_\theta P(\tau|\theta) R(\tau)$$

Why is this a problem?

- **Computationally useless:** You would have to calculate this for **every single possible future trajectory**. In any real environment, the number of trajectories is practically infinite.
- **It is no longer an expectation:** In ML, we bypass infinite sums by taking random samples and averaging them.
- An expectation *must* look like: $\sum P(x) \times$ [something].
- Right now, we just have $\sum \nabla P(x) \times$ [something]. We lost our standard probability weight $P(\tau|\theta)$ in the front.

Western UNIVERSITY · CANADA

The Rescue: The Log-Derivative Trick

We need to magically put $P(\tau|\theta)$ back into the formula so we can turn it back into an expectation and estimate it using random samples.

We do this using a basic rule of calculus: the derivative of a logarithm.

$$\nabla_\theta \log P(\tau|\theta) = \frac{\nabla_\theta P(\tau|\theta)}{P(\tau|\theta)}$$

If we rearrange this by multiplying both sides by $P(\tau|\theta)$, we get:

The Log-Derivative Identity

$$\nabla_\theta P(\tau|\theta) = P(\tau|\theta)\nabla_\theta \log P(\tau|\theta)$$

The Result: Back to Expectation

Now, we substitute this identity back into our problematic gradient equation:

$$\nabla_\theta J(\theta) = \sum_\tau P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau)$$

We have $\sum P(\tau|\theta) \times$ [something]. **It is an expectation again!**

The Policy Gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)]$$

Because it is an expectation, we no longer have to calculate every possible future. We can just drop an agent into the environment, let it play a few games (sample a few trajectories), measure the rewards, and average the results. The math trick simply reformats the equation so that sampling becomes mathematically valid.

Environment Cancellation Trick

Step 1: Deconstruct the Trajectory Probability

First, we must mathematically define what a trajectory τ actually is. It is a sequence of states and actions: $(s_0, a_0, s_1, a_1, \ldots, s_T, a_T)$.

Using the Markov property, the total probability of experiencing this specific trajectory, $P(\tau|\theta)$, is the product of three things:

1. The probability of starting in state s_0: $P(s_0)$
2. The probability of the agent choosing action a_t in state s_t: $\pi_\theta(a_t|s_t)$
3. The probability of the environment transitioning to state s_{t+1} given s_t and a_t: $P(s_{t+1}|s_t, a_t)$

Written as an equation:

$$P(\tau|\theta) = P(s_0) \prod_{t=0}^{T} \pi_\theta(a_t|s_t) P(s_{t+1}|s_t, a_t)$$

Environment Cancellation Trick

Step 2: Apply the Logarithm
In our gradient estimator, we don't have $P(\tau|\theta)$; we have $\log P(\tau|\theta)$ (thanks to the log-derivative trick).

A fundamental rule of logarithms is that the log of a product is the sum of the logs ($\log(A \times B) = \log(A) + \log(B)$). Let's apply this to our trajectory equation:

$$\log P(\tau|\theta) = \log P(s_0) + \sum_{t=0}^{T} [\log \pi_\theta(a_t|s_t) + \log P(s_{t+1}|s_t, a_t)]$$

Environment Cancellation Trick

Step 3: Take the Gradient

Now we apply our gradient ∇_θ to this expanded sum:

$$\nabla_\theta \log P(\tau|\theta) = \nabla_\theta \left[\log P(s_0) + \sum_{t=0}^{T} \log \pi_\theta(a_t|s_t) + \sum_{t=0}^{T} \log P(s_{t+1}|s_t, a_t) \right]$$

Since the environment's initial state and transition probabilities ($P(s_0)$ and $P(s_{t+1}|s_t, a_t)$) do not depend on your neural network parameters (θ), their derivatives are **zero**.

They completely disappear, leaving only the policy term:

The Simplified Trajectory Gradient

$$\nabla_\theta \log P(\tau|\theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Environment Cancellation Trick

Step 4: Back to the Expectation

Now we plug our simplified gradient back into the expectation we derived earlier:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\left(\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) R(\tau) \right]$$

This is a massive theoretical victory. This derivation proves that **model-free reinforcement learning** is mathematically sound and practically possible.

- We can calculate the exact gradient to update our network *without* needing a mathematical model of how the environment works.
- The complex transition probabilities $P(s_{t+1}|s_t, a_t)$ are entirely bypassed.
- The environment can be treated as a complete **black box**. We only need to observe states, take actions, and collect rewards!

Environment Cancellation Trick

Step 5: Linearity of Expectation

We currently have the expectation of a sum over the entire trajectory:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right]$$

Because the expectation operator is linear, the expectation of a sum is equal to the sum of the expectations. We can move the expectation *inside* the summation:

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T} \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right]$$

Why this matters:

By pulling the sum to the outside, we isolate the policy gradient at each individual timestep t. This isolation is the exact mathematical stepping stone we need to solve the credit assignment problem!

Credit Assignment Problem

Step 1: Splitting the Reward

Let's look at the term inside the expectation for a single time step t:

$$\nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)$$

We know that the total trajectory reward $R(\tau)$ is just the sum of all rewards from step 0 to T. We can split this into two parts: the **Past Rewards** (before a_t) and the **Future Rewards** (from a_t onwards).

$$R(\tau) = \underbrace{\sum_{t'=0}^{t-1} r_{t'}}_{\text{Past}} + \underbrace{\sum_{t'=t}^{T} r_{t'}}_{\text{Future}}$$

Credit Assignment Problem

If we substitute this split back into our gradient term, we can distribute the log policy gradient:

$$\left(\nabla_\theta \log \pi_\theta(a_t|s_t) \sum_{t'=0}^{t-1} r_{t'}\right) + \left(\nabla_\theta \log \pi_\theta(a_t|s_t) \sum_{t'=t}^{T} r_{t'}\right)$$

Our goal now is to prove that the expectation of the first term (the past rewards) is exactly zero.

Step 2: The "Score Function" Identity

To prove the past vanishes, we need a fundamental lemma from calculus and probability: **The expectation of the gradient of the log probability of any valid distribution is always zero.**

We can prove this using the definition of expectation and our log-derivative trick in reverse:

$$\mathbb{E}_{x\sim P_\theta}[\nabla_\theta \log P_\theta(x)] = \sum_x P_\theta(x)\nabla_\theta \log P_\theta(x)$$

$$= \sum_x P_\theta(x)\frac{\nabla_\theta P_\theta(x)}{P_\theta(x)}$$

$$= \sum_x \nabla_\theta P_\theta(x)$$

Step 2: The "Score Function" Proof (Cont.)

Continuing from the previous slide, our terms have canceled out to leave:

$$= \sum_x \nabla_\theta P_\theta(x)$$

Because the derivative of a sum is the sum of the derivatives (linearity), we can pull the gradient operator outside the summation:

$$= \nabla_\theta \sum_x P_\theta(x)$$

By definition, the sum of all probabilities in a distribution equals 1. And the derivative of the constant 1 is 0.

$$= \nabla_\theta(1) = 0$$

Step 3: Applying the Identity to the Past

Now, let's apply our identity to the first term from Step 1 (the past rewards). We want to take the expectation over the entire trajectory τ.

Using the Law of Iterated Expectations, we can isolate the expectation of taking action a_t at state s_t, conditional on everything that happened before it:

$$\mathbb{E}_\tau \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \sum_{t'=0}^{t-1} r_{t'} \right]$$

The Key Insight:
The past rewards ($\sum_{t'=0}^{t-1} r_{t'}$) were fully determined *before* we chose action a_t.

Step 3: The Past Vanishes

Because the past rewards are already fixed by the time we take action a_t, they are functionally just **constants** with respect to the current action.

Therefore, we can pull the past rewards out of the inner expectation over a_t:

$$= \mathbb{E}_{\text{past}} \left[\left(\sum_{t'=0}^{t-1} r_{t'} \right) \underbrace{\mathbb{E}_{a_t \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t | s_t) \right]}_{\text{This equals 0!}} \right]$$

Step 4: Dropping the Past

Since the expectation of the past rewards term is exactly zero, we can safely delete it from our gradient equation entirely.

This leaves us only with the second term—the sum of **future rewards** (often denoted as the Reward-to-Go $\hat{R}_t$ or the action-value $Q(s_t, a_t)$):

The Reward-to-Go Policy Gradient

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T} \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \sum_{t'=t}^{T} r_{t'} \right]$$

The Benefit:
This mathematical proof allows us to confidently drop the past. By doing so, we remove a massive source of noise, which significantly reduces the variance of our policy updates!

Variance Reduction & The Advantage Function

The Problem: High Variance

- Raw future returns ($\hat{R}_t$) can be massive (e.g., always $+1000$ to $+1050$).
- Scaling gradients by huge, strictly positive numbers causes unstable, bouncing training updates.

The Solution: A State-Dependent Baseline

- Mathematically, we can subtract a baseline from the return as long as it **does not depend on the action** a_t.
- Subtracting the expected State-Value $V_\phi(s_t)$ centers the values around zero without changing the expected direction of the gradient.

The Empirical Advantage Estimator

$$\hat{A}_t = \left(\sum_{t'=t}^{T} \gamma^{t'-t} r_{t'} \right) - V_\phi(s_t)$$

**Note: We introduce the discount factor γ here to prioritize near-term rewards over distant ones.*

The Complete Policy Gradient

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T} \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \hat{A}_t \right]$$

Intuition: How it updates the network

- If the advantage $\hat{A}_t > 0$ (the action was better than average), the gradient pushes θ to make taking action a_t in state s_t **more** likely.
- If the advantage $\hat{A}_t < 0$ (the action was worse than average), the gradient pushes θ to make taking that action **less** likely.

Changing Perspective: Trajectories to States

Currently, our expectation is over the entire trajectory ($\tau \sim \pi_\theta$). But let's look closely at the terms inside the expectation for a specific time step t:

$$\nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t$$

The Key Insight:
These terms *only* depend on the current state s_t and the current action a_t. They no longer care about the rest of the trajectory!

Therefore, we can rewrite the expectation over full trajectories as an expectation over the specific states and actions encountered at time t:

State-Action Expectation

$$\nabla_\theta J(\theta) = \sum_{t=0}^{T} \mathbb{E}_{s_t, a_t \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t)\hat{A}_t \right]$$

Absorbing the Sum: The Practical Shift

This is where the PPO paper makes its jump from theory to practice.

How it is actually coded:

- We do not calculate the gradient by looping over time steps one by one and calculating an expectation for each.
- Instead, we run our agent through the environment for N steps to collect a massive batch of individual experiences: (s, a, r, s').
- We throw all of these individual time steps into a giant buffer.
- When it is time to update the network, we just take an **average** over all the independent time steps in that batch.

Absorbing the Sum: The Notation Shift

Mathematically, taking an average over time steps is written as: $\frac{1}{T}\sum_{t=0}^{T}$.

Can we just divide our objective by T? **Yes.** Scaling an objective function by a positive constant does not change the location of its maximum.

By dividing by T, the summation over t literally becomes an **empirical expectation** over the time steps in our batch:

The Notation Shift

$$\frac{1}{T}\sum_{t=0}^{T}\dots \quad \Rightarrow \quad \hat{\mathbb{E}}_t[\dots]$$

Note: In PPO papers, $\hat{\mathbb{E}}_t$ indicates the empirical average over a finite batch of samples, alternating between sampling and optimization.

References